管理及保護應用程式

Astra Control Service

NetApp
June 28, 2022

This PDF was generated from https://docs.netapp.com/zh-tw/astra-control-service/use/manage-apps.html on June 28, 2022. Always check docs.netapp.com for the latest.

目錄

管理及保護應用程式 1
開始管理應用程式 1
利用快照與備份來保護應用程式 6
還原應用程式 9
複製及移轉應用程式 9
管理應用程式執行掛勾 11

管理及保護應用程式

開始管理應用程式

您先請 "將Kubernetes叢集新增至Astra Control"、您可以在叢集上安裝應用程式（Astra Control之外）、然後前往Astra Control的「應用程式」頁面、開始管理應用程式。

應用程式管理需求

請考慮下列Astra Control應用程式管理需求：

- 授權：若要使用Astra Control Center管理應用程式、您需要Astra Control Center授權。
- 命名空間：Astra Control要求應用程式不超過一個命名空間、但命名空間可以包含多個應用程式。
- * StorageClass *：如果您安裝的應用程式已明確設定StorageClass、且需要複製應用程式、則複製作業的目標叢集必須具有原本指定的StorageClass。將具有明確設定StorageClass的應用程式複製到沒有相同StorageClass的叢集、將會失敗。
- * Kubernetes資源*：使用未由Astra Control收集的Kubernetes資源的應用程式、可能沒有完整的應用程式資料管理功能。Astra Control會收集下列Kubernetes資源：

叢集角色	ClusterRoleBinding	組態對應
CustomResourceDesDefinition	CustomResource	可關係工作
示範	HorizontalPodAutoscaler	入侵
部署組態	互鎖Webhook	PeristentVolume Claim
Pod	Podcast中斷預算	Podcast範本
網路原則	ReplicaSet	角色
角色繫結	路由	秘密
服務	服務帳戶	狀態集
驗證Webhook		

支援的應用程式安裝方法

您可以使用下列方法安裝想要使用Astra Control管理的應用程式：

- 資訊清單檔案：Astra Control支援使用KUbectl從資訊清單檔案安裝的應用程式。例如：

```
kubectl apply -f myapp.yaml
```

- * Helm 3*：如果您使用Helm來安裝應用程式、Astra Control需要Helm版本3。完全支援使用Helm 3（或從Helm 2升級至Helm 3）來管理及複製應用程式。不支援管理以Helm 2安裝的應用程式。
- 操作員部署的應用程式：Astra Control支援以命名空間範圍運算子安裝的應用程式。這些運算子通常採用「傳遞值」而非「傳遞參照」架構來設計。以下是一些遵循這些模式的營運者應用程式：

- "Apache K8ssandra"

> 對於k8ssandra、支援就地還原作業。若要還原新命名空間或叢集的作業、必須先關閉應用程式的原始執行個體。這是為了確保傳遞的對等群組資訊不會導致跨執行個體通訊。不支援複製應用程式。

- "Jenkins CI"
- "Percona XtraDB叢集"

請注意、Astra Control可能無法複製以「傳遞參考」架構設計的操作員（例如、CockroachDB操作員）。在這些類型的複製作業中、複製的操作員會嘗試從來源操作員參考Kubernetes機密、儘管在複製程序中有自己的新秘密。由於Astra Control不知道來源營運者的Kubernetes機密資料、因此複製作業可能會失敗。

> 運算子及其安裝的應用程式必須使用相同的命名空間；您可能需要修改運算子的部署.yaml檔案、以確保情況如此。

在叢集上安裝應用程式

現在您已將叢集新增至Astra Control、即可在叢集上安裝應用程式。在預設情況下、會在新的儲存類別上配置持續磁碟區。在Pod上線後、您可以使用Astra Control來管理應用程式。

Astra Control只有在儲存設備位於由Astra Control安裝的儲存類別時、才會管理狀態化應用程式。

- "深入瞭解GKE叢集的儲存課程"
- "深入瞭解適用於高效能叢集的儲存類別"

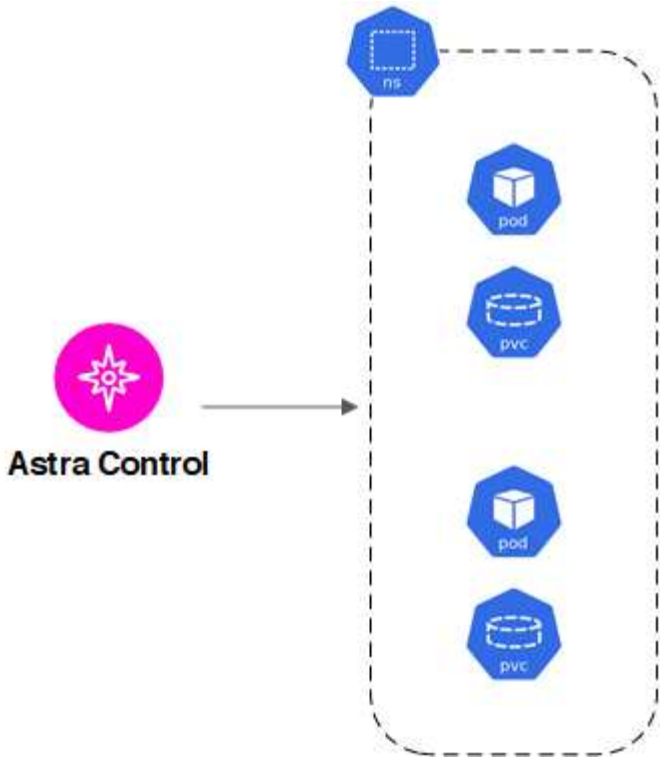
如需從Helm圖表部署一般應用程式的說明、請參閱下列內容：

- "從Helm圖表部署MariaDB"
- "從Helm圖表部署MySQL"
- "從Helm圖表部署Postgres"
- "從Helm圖表部署Jenkins"

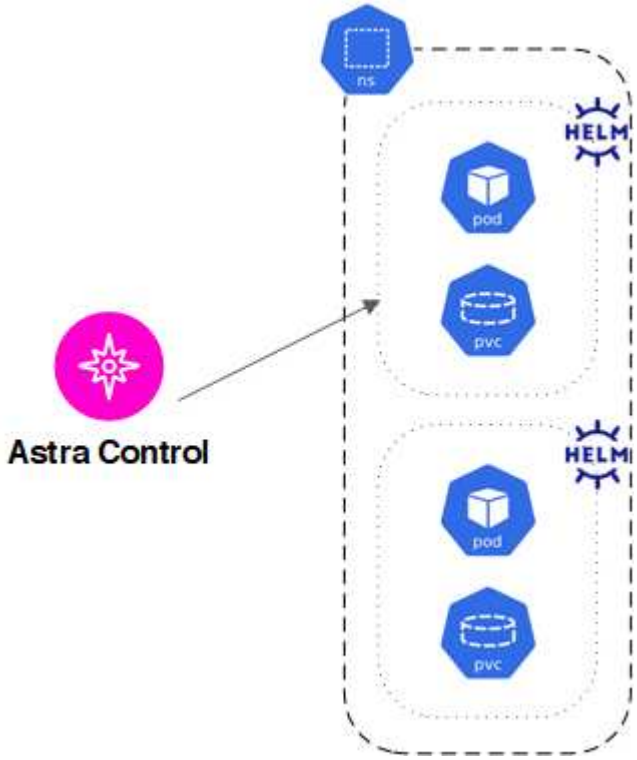
管理應用程式

當Astra Control探索叢集上執行的應用程式時、這些應用程式會處於不受管理的狀態、直到您選擇要管理的方式為止。Astra Control中的託管應用程式可以是下列任一項：

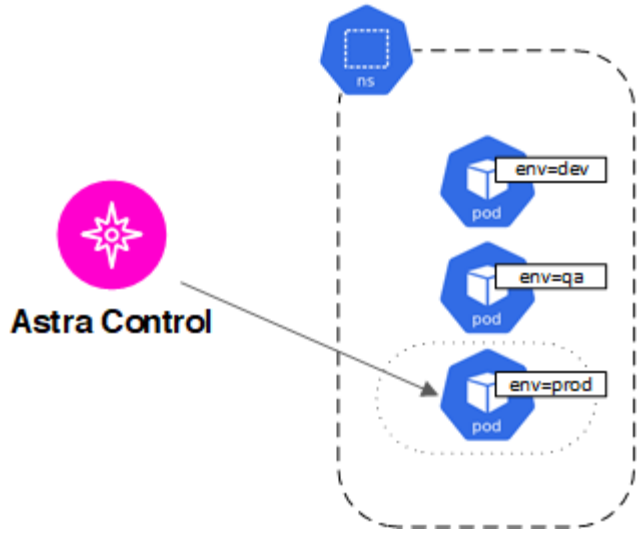
- 命名空間、包括該命名空間中的所有資源



- 在命名空間內部署有helm3的個別應用程式



- 一組資源、由Kubernetes標籤（在Astra Control中稱為_custom app_）在命名空間內識別



以下各節將說明如何使用這些選項來管理應用程式。

依命名空間管理應用程式

「應用程式」頁面的「探索到」區段會顯示這些命名空間中的命名空間、以及已安裝Helm的應用程式或自訂標記的應用程式。您可以選擇個別或在命名空間層級管理每個應用程式。所有這些都達到資料保護作業所需的精細度。

例如、您可能想要設定每週執行時間的「MARIA」備份原則、但您可能需要比這更頻繁地備份「MariaDB」（位於同一個命名空間）。根據這些需求、您需要分別管理應用程式、而非在單一命名空間下管理。

雖然Astra Control可讓您分別管理階層的兩個層級（命名空間和命名空間中的應用程式）、但最佳實務做法是選擇一個或另一個層級。如果在命名空間和應用程式層級同時執行動作、則Astra Control中所採取的動作可能會失敗。

步驟

1. 選取*應用程式*、然後選取*已探索*。
2. 檢視探索到的命名空間清單、然後展開命名空間以檢視應用程式和相關資源。

   Astra Control會在命名空間中顯示Helm應用程式和自訂標記的應用程式。如果有Helm標籤、則會以標籤圖示標示。

3. 決定要個別管理每個應用程式、還是在命名空間層級管理。
4. 在階層中所需的層級、選取* Actions （動作）欄中的下拉式清單、然後選取* Manage （管理）。
5. 如果您不想管理應用程式、請在所需應用程式的*「Actions」（動作）欄中選取下拉式清單、然後選取「Ignori*」（忽略）。

   例如、如果您想要一起管理「Jenkins」命名空間下的所有應用程式、使其具有相同的快照和備份原則、您可以管理命名空間並忽略命名空間中的應用程式。

您選擇管理的應用程式現在可從*「託管」*索引標籤取得。任何忽略的應用程式都會移至*忽略*索引標籤。理想情況下、探索到的索引標籤會顯示零應用程式、以便在安裝新應用程式時、更容易找到及管理。

依**Kubernetes**標籤管理應用程式

Astra Control在應用程式頁面頂端包含一個名為*定義自訂應用程式*的動作。您可以使用此動作來管理以Kubernetes標籤識別的應用程式。 "深入瞭解如何透過Kubernetes標籤定義應用程式"。

步驟

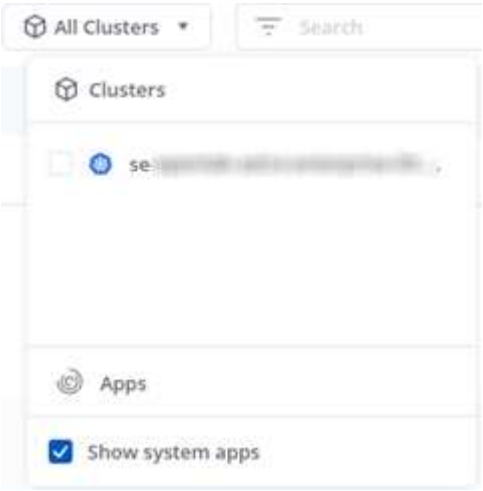
1. 選擇*應用程式>定義自訂應用程式*。
2. 在「定義自訂應用程式」對話方塊中、提供管理應用程式所需的資訊：
   a. 新應用程式：輸入應用程式的顯示名稱。
   b. 叢集：選取應用程式所在的叢集。
   c. 命名空間：選取應用程式的命名空間。
   d. 標籤：輸入標籤或從下列資源中選取標籤。
   e. 選取的資源：檢視及管理您要保護的選定Kubernetes資源（Pod、機密、持續磁碟區等）。
      - 展開資源並選取標籤數量、即可檢視可用的標籤。
      - 選取其中一個標籤。

        選擇標籤後、標籤會顯示在*標籤*欄位中。Astra Control也會更新*未選取的資源*區段、以顯示與所選標籤不符的資源。
   f. 未選取的資源：確認您不想保護的應用程式資源。
3. 選擇*定義自訂應用程式*。

Astra Control可管理應用程式。您現在可以在*託管*索引標籤中找到它。

系統應用程式呢？

Astra Control也會探索Kubernetes叢集上執行的系統應用程式。您可以篩選應用程式清單來檢視這些項目。



我們預設不會顯示這些系統應用程式、因為您很少需要備份這些應用程式。

利用快照與備份來保護應用程式

使用自動保護原則或臨機操作、拍攝快照和備份資料、保護您的應用程式。

快照與備份

_snapshot是應用程式的時間點複本、儲存在與應用程式相同的已配置磁碟區上。通常速度很快。本機快照可用來將應用程式還原至較早的時間點。

備份_儲存在雲端的物件儲存設備上。相較於本機快照、備份速度可能較慢。但可在雲端跨區域存取、以進行應用程式移轉。您也可以選擇較長的備份保留期間。

> 您必須等到最近進行備份之後、才能獲得完整保護。這很重要、因為備份儲存在遠離持續磁碟區的物件存放區中。如果故障或意外損壞叢集及其持續儲存設備、則需要備份才能恢復。快照無法讓您恢復。

設定保護原則

保護原則可在已定義的排程中建立快照、備份或兩者、以保護應用程式。您可以選擇每小時、每天、每週和每月建立快照和備份、也可以指定要保留的複本數量。如果您需要每小時執行一次以上的備份或快照、您可以 "使用Astra Control REST API建立快照與備份"。

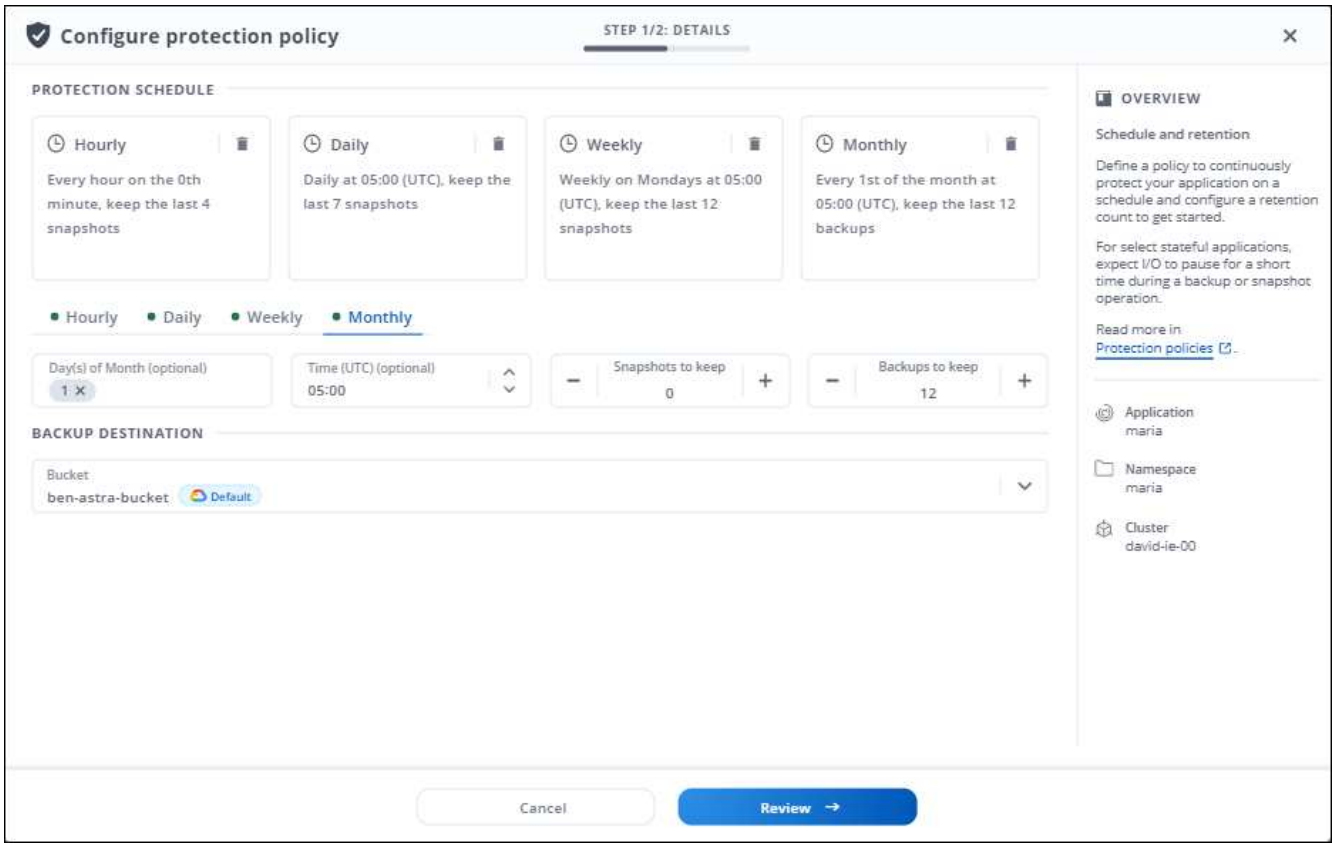
步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。
3. 選取*設定保護原則*。
4. 選擇每小時、每日、每週和每月排程所需保留的快照和備份數量、以定義保護排程。

   您可以同時定義每小時、每日、每週及每月排程。在您設定快照和備份的保留層級之前、排程不會變成作用中。

   當您設定備份的保留層級時、可以選擇要儲存備份的儲存區。

   下列範例設定四種保護排程：每小時、每日、每週及每月提供快照與備份。



5. 選擇* Review *。
6. 選擇*設定*。

   以下是每個步驟的影片。

   ▶ https://docs.netapp.com/zh-tw/astra-control-service/media/use/video-set-protection-policy.mp4 *(video)*

Astra Control會使用您定義的排程和保留原則來建立和保留快照和備份、以實作資料保護原則。

建立快照

您可以隨時建立隨需快照。

步驟

1. 選擇*應用程式*。
2. 選取所需應用程式*「Actions」（動作）*欄中的下拉式清單。
3. 選擇* Snapshot *。
4. 自訂快照名稱、然後選取*檢閱資訊*。
5. 檢閱快照摘要、然後選取* Snapshot App*。

Astra Control會建立應用程式的快照。

建立備份

您也可以隨時備份應用程式。

步驟

1. 選擇*應用程式*。
2. 選取所需應用程式*「Actions」（動作）*欄中的下拉式清單。
3. 選擇*備份*。
4. 自訂備份名稱、選擇是否要從現有的快照備份應用程式、然後選取*檢閱資訊*。
5. 檢閱備份摘要、然後選取*備份應用程式*。

Astra Control會建立應用程式的備份。

檢視快照與備份

您可以從「資料保護」索引標籤檢視應用程式的快照與備份。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。

   快照預設會顯示。

3. 選取*備份*以查看備份清單。

刪除快照

刪除不再需要的排程或隨需快照。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。
3. 在* Actions（操作）*列中選擇所需快照的下拉列表。
4. 選擇*刪除snapshot *。
5. 輸入快照名稱以確認刪除、然後選取*是、刪除快照*。

Astra Control會刪除快照。

刪除備份

刪除不再需要的排程或隨需備份。

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。

3. 選擇*備份*。
4. 在* Actions（操作）*列中選擇下拉列表以進行所需的備份。
5. 選擇*刪除備份*。
6. 輸入備份名稱以確認刪除、然後選擇*是、刪除備份*。

Astra Control會刪除備份。

還原應用程式

Astra Control可以從快照或備份還原應用程式。將應用程式還原至同一個叢集時、從現有的快照還原速度會更快。您可以使用Astra Control UI或 "Astra Control API" 以還原應用程式。

> 任何具有命名空間名稱/ ID或命名空間標籤限制的成員使用者、都可以將應用程式複製或還原到同一個叢集上的新命名空間、或是組織帳戶中的任何其他叢集。不過、相同的使用者無法存取新命名空間中的複製或還原應用程式。在複製或還原作業建立新命名空間之後、帳戶管理員/擁有者可以編輯成員使用者帳戶、並更新受影響使用者的角色限制、以便授予新命名空間的存取權。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選擇*資料保護*。
3. 如果您想要從快照還原、請保持選取* Snapshot*。否則、請選取*備份*以從備份還原。
4. 在* Actions （操作）欄中選取您要還原的快照或備份下拉式清單。
5. 選取*還原應用程式*。
6. 還原詳細資料：指定還原應用程式的詳細資料。若要就地還原應用程式（將應用程式還原為本身的舊版）、請選擇目前執行的相同命名空間和目的地叢集：
   - 輸入應用程式的命名空間。
   - 選擇應用程式的目的地叢集。
   - 按一下* Review *。

   > 如果還原至先前刪除的命名空間、則會在還原程序中建立名稱相同的新命名空間。任何在先前刪除命名空間中擁有管理應用程式權限的使用者、都必須手動還原新重新建立命名空間的權限。

7. 還原摘要：檢閱還原動作的詳細資料、輸入「還原」、然後選取*還原*。

Astra Control會根據您提供的資訊還原應用程式。如果您就地還原應用程式、則會刪除並重新建立相關的持續磁碟區。

複製及移轉應用程式

複製現有的應用程式、在相同的Kubernetes叢集或其他叢集上建立複製的應用程式。如果您需要將應用程式和儲存設備從一個Kubernetes叢集移至另一個叢集、複製作業將有助於

您。例如、您可能想要透過CI/CD傳輸途徑和Kubernetes命名空間來移動工作負載。

當Astra Control複製應用程式時、會建立應用程式組態和持續儲存的複本。

- 如果您複製由操作人員部署的Jenkins CI執行個體、則需要手動還原持續性資料。這是應用程式部署模式的限制。
- 如果您部署的應用程式已明確設定StorageClass、且需要複製應用程式、則目標叢集必須具有原本指定的StorageClass。將具有明確設定StorageClass的應用程式複製到沒有相同StorageClass的叢集、將會失敗。
- 在應用程式備份或應用程式還原期間、您可以選擇性地指定庫位ID。不過、應用程式複製作業一律會使用已定義的預設儲存區。沒有選項可變更實體複本的儲存區。如果您想要控制所使用的儲存桶、您也可以選擇 "變更庫位預設值" 或執行 "備份" 接著是A "還原" 獨立提供。
- 任何具有命名空間名稱/ ID或命名空間標籤限制的成員使用者、都可以將應用程式複製或還原到同一個叢集上的新命名空間、或是組織帳戶中的任何其他叢集。不過、相同的使用者無法存取新命名空間中的複製或還原應用程式。在複製或還原作業建立新命名空間之後、帳戶管理員/擁有者可以編輯成員使用者帳戶、並更新受影響使用者的角色限制、以便授予新命名空間的存取權。

步驟

1. 選擇*應用程式*。
2. 選取所需應用程式*「Action"（行動）」欄中的下拉式清單。
3. 選擇* Clone（克隆）*。
4. 複製詳細資料：指定複製的詳細資料：
   - 保留預設名稱和命名空間、或加以編輯。
   - 選擇要複製的目的地叢集。
   - 選擇是要從現有的快照或備份建立複本。如果您未選取此選項、Astra Control會從應用程式的目前狀態建立複本。
5. * Clone Summary（複製摘要）：檢閱有關複製的詳細資料、然後選取 Clone App*（複製應用程式*）。

Astra Control會根據您提供的資訊來複製該應用程式。

管理應用程式執行掛勾

執行攔截是自訂指令碼、您可以在託管應用程式的快照之前或之後執行。例如、如果您有資料庫應用程式、您可以使用執行掛勾來暫停快照之前的所有資料庫交易、並在快照完成後繼續交易。如此可確保應用程式一致的快照。

預設執行掛勾和規則運算式

對於某些應用程式、Astra Control隨附NetApp提供的預設執行掛勾、可處理快照前後的凍結和解凍作業。Astra Control使用規則運算式、將應用程式的容器映像與下列應用程式配對：

- MariaDB
  - 符合規則運算式：\bmariadb\b
- MySQL
  - 相符的規則運算式：\bmysql\b
- PostgreSQL
  - 相符的規則運算式：\bpostgresql\b

如果有相符項目、則NetApp提供的該應用程式預設執行掛勾會顯示在應用程式的作用中執行掛勾清單中、而這些掛勾會在擷取該應用程式的快照時自動執行。如果其中一個自訂應用程式有類似的映像名稱、正好符合其中一個規則運算式（而且您不想使用預設的執行掛勾）、您可以變更映像名稱、 或停用該應用程式的預設執行掛勾、改用自訂掛勾。

您無法刪除或修改預設的執行掛勾。

關於自訂執行掛勾的重要注意事項

規劃應用程式的執行掛勾時、請考量下列事項。

- Astra Control需要以執行Shell指令碼的格式寫入執行掛勾。
- 指令碼大小限制為128 KB。
- Astra Control會使用執行掛勾設定和任何符合條件來判斷哪些掛勾適用於快照。
- 所有執行掛機故障都是軟性故障、即使掛機故障、仍會嘗試其他掛機和快照。但是、當掛機失敗時、會在*活動*頁面事件記錄中記錄警告事件。
- 若要建立、編輯或刪除執行掛勾、您必須是擁有擁有者、管理員或成員權限的使用者。
- 如果執行掛機執行時間超過25分鐘、掛機將會失敗、並建立傳回代碼為「N/A」的事件記錄項目。任何受影響的快照都會逾時並標示為故障、並會出現一個事件記錄項目、指出逾時時間。

> 由於執行掛勾通常會減少或完全停用執行中應用程式的功能、因此您應該一律盡量縮短自訂執行掛勾執行所需的時間。

執行快照時、執行掛機事件會依照下列順序進行：

1. 任何適用的NetApp提供的預設快照前執行掛勾、都會在適當的容器上執行。
2. 任何適用的自訂快照前執行掛勾都會在適當的容器上執行。您可以視需要建立及執行任意數量的自訂快照前

置掛勾、但在快照之前執行這些掛勾的順序並不保證也無法設定。

3. 快照即會執行。
4. 任何適用的自訂快照後執行掛勾都會在適當的容器上執行。您可以視需要建立及執行任意數量的自訂快照後掛勾、但快照後執行這些掛勾的順序並不保證也無法設定。
5. 任何適用的NetApp提供的預設快照後執行掛勾、都會在適當的容器上執行。

> 在正式作業環境中啟用執行攔截指令碼之前、請務必先進行測試。您可以使用'kubecl exec'命令來方便地測試指令碼。在正式作業環境中啟用執行掛勾之後、請測試所產生的快照、以確保它們一致。您可以將應用程式複製到暫用命名空間、還原快照、然後測試應用程式、藉此完成此作業。

檢視現有的執行掛勾

您可以檢視現有的自訂或NetApp提供的應用程式預設執行勾點。

步驟

1. 移至*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。

   您可以在結果清單中檢視所有已啟用或已停用的執行掛勾。您可以查看Hook的狀態、來源、以及何時執行（快照前或快照後）。若要檢視執行掛起的相關事件記錄、請前往左側導覽區域的*活動*頁面。

建立自訂執行掛勾

您可以為應用程式建立自訂執行掛勾。請參閱 "執行攔截範例" 如需攔截範例、您需要擁有擁有擁有者、管理員或成員權限、才能建立執行掛勾。

> 當您建立自訂Shell指令碼作為執行掛勾時、請記得在檔案開頭指定適當的Shell、除非您執行Linux命令或提供執行檔的完整路徑。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。
3. 選取*新增連結*。
4. 在* HookDetails（掛機詳細資料）區域中、視掛機的執行時間而定、選擇*預先快照*或*快照後。
5. 輸入掛機的唯一名稱。
6. （選用）輸入執行期間要傳遞至掛機的任何引數、並在您輸入的每個引數之後按Enter鍵以記錄每個引數。
7. 在「* Container images"（* Container映像*）區域中、如果掛勾應針對應用程式中包含的所有容器映像執行、請啟用「* Apply to all Container images"（套用至所有容器映像）核取方塊。如果掛機只能對一個或多個指定的容器映像起作用、請在「要比對的容器映像名稱」欄位中輸入容器映像名稱。
8. 在*指令碼*區域中、執行下列其中一項：
   - 上傳自訂指令碼。
     i. 選取*上傳檔案*選項。

    ii. 瀏覽至檔案並上傳。
    iii. 為指令碼指定唯一名稱。
    iv. （選用）輸入其他系統管理員應該知道的任何指令碼附註。
  - 從剪貼簿貼入自訂指令碼。
    i. 選擇*從剪貼簿貼上*選項。
    ii. 選取文字欄位、然後將指令碼文字貼到欄位中。
    iii. 為指令碼指定唯一名稱。
    iv. （選用）輸入其他系統管理員應該知道的任何指令碼附註。
9. 選取*新增攔截*。

停用執行掛勾

如果您想要暫時避免在應用程式快照之前或之後執行、可以停用執行掛勾。您需要擁有擁有擁有者、管理員或成員權限、才能停用執行掛勾。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。
3. 選取*「動作*」下拉式清單以顯示您要停用的掛勾。
4. 選擇*停用*。

刪除執行掛勾

如果不再需要執行掛勾、您可以完全移除該掛勾。您需要擁有擁有擁有者、管理員或成員權限、才能刪除執行掛勾。

步驟

1. 選取*應用程式*、然後選取託管應用程式的名稱。
2. 選取*執行掛勾*索引標籤。
3. 選取*「動作」*下拉式選單以選取您要刪除的掛勾。
4. 選擇*刪除*。

執行攔截範例

請使用下列範例、瞭解如何建立執行掛勾的架構。您可以使用這些hooks做為範本、或做為測試指令碼。

簡單的成功範例

這是一個簡單的勾點、可成功將訊息寫入標準輸出和標準錯誤。

```
#!/bin/sh
```

```
# success_sample.sh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.sh"

# exit with 0 to indicate success
```

```
info "exit 0"
exit 0
```

簡單的成功範例（**Bash**版本）

這是一個簡單的攔截、成功將訊息寫入標準輸出和標準錯誤（寫入Bash）。

```
#!/bin/bash

# success_sample.bash
#
# A simple noop success hook script for testing purposes.
#
# args: None


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
```

```
# main
#

# log something to stdout
info "running success_sample.bash"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

簡單的成功範例（**zsh**版本）

這是一個簡單的勾點範例、可成功將訊息寫入標準輸出和標準錯誤、並寫入Z Shell。

```
#!/bin/zsh

# success_sample.zsh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
```

```
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}

#
# main
#

# log something to stdout
info "running success_sample.zsh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

成功的引數範例

下列範例示範如何在攔截中使用args。

```
#!/bin/sh

# success_sample_args.sh
#
# A simple success hook script with args for testing purposes.
#
# args: Up to two optional args that are echoed to stdout

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
```

```
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample_args.sh"


# collect args
arg1=$1
arg2=$2

# output args and arg count to stdout
info "number of args: $#"
info "arg1 ${arg1}"
info "arg2 ${arg2}"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

快照前/快照後掛機範例

以下範例說明如何將相同的指令碼同時用於快照前和快照後掛勾。

```
#!/bin/sh

# success_sample_pre_post.sh
#
# A simple success hook script example with an arg for testing purposes
# to demonstrate how the same script can be used for both a prehook and
posthook
#
# args: [pre|post]
```

```
# unique error codes for every error case
ebase=100
eusage=$((ebase+1))
ebadstage=$((ebase+2))
epre=$((ebase+3))
epost=$((ebase+4))


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# Would run prehook steps here
#
prehook() {
    info "Running noop prehook"
    return 0
}
```

```
#
# Would run posthook steps here
#
posthook() {
    info "Running noop posthook"
    return 0
}


#
# main
#

# check arg
stage=$1
if [ -z "${stage}" ]; then
    echo "Usage: $0 <pre|post>"
    exit ${eusage}
fi

if [ "${stage}" != "pre" ] && [ "${stage}" != "post" ]; then
    echo "Invalid arg: ${stage}"
    exit ${ebadstage}
fi

# log something to stdout
info "running success_sample_pre_post.sh"

if [ "${stage}" = "pre" ]; then
    prehook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during prehook"
    fi
fi

if [ "${stage}" = "post" ]; then
    posthook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during posthook"
    fi
fi

exit ${rc}
```

故障範例

下列範例示範如何處理攔截式故障。

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#
```

```
# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

詳細故障範例

下列範例示範如何以更詳細的記錄功能來處理掛機中的失敗。

```
#!/bin/sh

# failure_sample_verbose.sh
#
# A simple failure hook script with args for testing purposes.
#
# args: [The number of lines to output to stdout]


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
```

```
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_verbose.sh"


# output arg value to stdout
linecount=$1
info "line count ${linecount}"

# write out a line to stdout based on line count arg
i=1
while [ "$i" -le ${linecount} ]; do
    info "This is line ${i} from failure_sample_verbose.sh"
    i=$(( i + 1 ))
done

error "exiting with error code 8"
exit 8
```

退出程式碼範例失敗

下列範例示範攔截失敗、並顯示結束程式碼。

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#


#
# Writes the given message to standard output
```

```
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

失敗後成功範例

下列範例示範第一次執行時發生掛機故障、但在第二次執行後成功。

```
#!/bin/sh

# failure_then_success_sample.sh
#
# A hook script that fails on initial run but succeeds on second run for
testing purposes.
#
# Helpful for testing retry logic for post hooks.
#
# args: None
#

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
```

```
info "running failure_success sample.sh"


if [ -e /tmp/hook-test.junk ] ; then
    info "File does exist.  Removing /tmp/hook-test.junk"
    rm /tmp/hook-test.junk
    info "Second run so returning exit code 0"
    exit 0
else
    info "File does not exist.  Creating /tmp/hook-test.junk"
    echo "test" > /tmp/hook-test.junk
    error "Failed first run, returning exit code 5"
    exit 5
fi
```

版權資訊

Copyright©2022 NetApp、Inc.版權所有。美國印製本文件中版權所涵蓋的任何部分、不得以任何形式或任何方式（包括影印、錄製、 在未事先取得版權擁有者書面許可的情況下、在電子擷取系統中進行錄音或儲存。

衍生自受版權保護之NetApp資料的軟體必須遵守下列授權與免責聲明：

本軟體係由NetApp「依現狀」提供、不含任何明示或暗示的保證、包括但不限於適售性及特定用途適用性的暗示保證、特此聲明。在任何情況下、NetApp均不對任何直接、間接、偶發、特殊、示範、 或衍生性損害（包括但不限於採購替代商品或服務；使用損失、資料或利潤損失； 或業務中斷）、無論是在合約、嚴格責任或侵權行為（包括疏忽或其他）中、無論是因使用本軟體而產生的任何責任理論（包括疏忽或其他）、即使已被告知可能造成此類損害。

NetApp保留隨時變更本文所述之任何產品的權利、恕不另行通知。除非NetApp以書面明確同意、否則NetApp不承擔因使用本文所述產品而產生的任何責任或責任。使用或購買本產品並不代表NetApp擁有任何專利權利、商標權利或任何其他智慧財產權。

本手冊所述產品可能受到一或多個美國國家/地區的保護專利、國外專利或申請中。

限制權利圖例：政府使用、複製或揭露受DFARS 252.277-7103（1988年10月）和FAR 52-227-19（1987年6月）技術資料與電腦軟體權利條款（c）（1）（ii）分段所述限制。

商標資訊

NetApp、NetApp標誌及所列的標章 http://www.netapp.com/TM 為NetApp、Inc.的商標。其他公司和產品名稱可能為其各自所有者的商標。